

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory work 3:

Study and Empirical Analysis of DFS and BFS Graph Traversal Algorithms

Elaborated:

st. gr. FAF-241

Pleșu Dinu

Verified:

asist. univ.

Fiștic Cristofor

TABLE OF CONTENTS

LIST OF ABBREVIATIONS AND DEFINITIONS	3
INTRODUCTION	4
1 ALGORITHM ANALYSIS.....	5
2 IMPLEMENTATION	7
2.1 Depth First Search	7
2.2 Breadth First Search	8
CONCLUSION.....	11
REFERENCES	12

LIST OF ABBREVIATIONS AND DEFINITIONS

- BFS - Breadth First Search, a graph traversal algorithm that explores vertices by levels;
- DFS - Depth First Search, a graph traversal algorithm that explores one branch before backtracking;
- graph density - ratio between existing edges and the maximum possible number of edges;
- KB - kilobytes;
- ms - milliseconds;
- traversal - process of visiting reachable vertices in a graph.

INTRODUCTION

Depth First Search and Breadth First Search are two fundamental graph traversal algorithms. Both determine which vertices are reachable from a starting point, but they explore the graph in different ways. DFS follows a branch as deeply as possible before backtracking, while BFS explores the graph level by level.

These traversal strategies are important because they form the basis for many other graph algorithms. In practice, their empirical behavior depends not only on the number of vertices, but also on the number of edges, on graph density, and on the implementation choices used in the program.

1 ALGORITHM ANALYSIS

Objective

Study and analyze Depth First Search and Breadth First Search on graphs with different sizes and structures.

Tasks

1. implement the algorithms listed above in a programming language;
2. establish the properties of the input data against which the analysis is performed;
3. choose metrics for comparing algorithms;
4. perform empirical analysis of the proposed algorithms;
5. make a graphical presentation of the data obtained;
6. make a conclusion on the work done.

Theoretical Notes

An alternative to mathematical analysis of complexity is empirical analysis. This may be useful for obtaining preliminary information on the complexity class of an algorithm, for comparing the efficiency of several algorithms that solve the same problem, and for observing the practical effect of implementation details on a particular computer [1, 2].

In the empirical analysis of an algorithm, the following steps are usually followed: the purpose of the analysis is established; the efficiency metric is selected; the properties of the input data are defined; the algorithms are implemented; multiple input sets are generated; the program is executed for each input set; the obtained data are analyzed and represented graphically.

Both DFS and BFS have theoretical time complexity $O(V+E)$ when the graph is represented through adjacency lists. Even so, the practical running time depends strongly on the number of edges, on the graph topology, and on the internal data structures used by the implementation [1, 2, 3].

Comparison Metric

The comparison metrics chosen for the laboratory were:

- execution time in milliseconds;
- peak memory usage in kilobytes;
- number of visited vertices;
- success rate of the execution.

Input Format

The laboratory used two complementary styles of graph generation:

- graphs classified by fill percentage, from 0% to 100%;
- graphs classified by topology, such as tree, complete graph, bipartite graph, cyclic graph, wheel graph, grid-planar graph, and disconnected graph.

For the base density program, representative sizes of 10, 50, 100, and 200 vertices were used. For the optimized topology program, sizes up to 1000 vertices were included in order to observe the scaling behavior more clearly.

2 IMPLEMENTATION

2.1 Depth First Search

The DFS implementation was prepared in both base and optimized forms. The base form uses simpler list-based structures, while the optimized form uses more suitable containers and membership tests. DFS explores one branch of the graph as deeply as possible before backtracking.

Algorithm Description

The DFS method follows the idea shown in the next pseudocode.

```
DFS(graph, start):
    mark start as visited
    push start on stack
    while stack is not empty:
        node = pop stack
        for each neighbor of node:
            if neighbor is not visited:
                mark neighbor as visited
                push neighbor on stack
```

Implementation

The project contains both a classic and an optimized DFS implementation in the `Lab3` folder, and these were benchmarked against multiple graph families [4].

Results

After running the DFS function for each graph size defined in the density and topology specifications and recording execution time and memory usage, the representative results are shown in Table 2.1.

Table 2.1 – DFS and BFS base results at 50% fill

Algorithm	Nodes	Time (ms)	Memory (KB)
BFS_Base	10	0.029	0.60
BFS_Base	50	0.288	1.32
BFS_Base	100	2.750	2.41
BFS_Base	200	35.760	4.29
DFS_Base	10	0.044	0.77
DFS_Base	50	0.453	6.45
DFS_Base	100	4.742	22.08
DFS_Base	200	30.207	86.64

In Table 2.1 it can be observed that DFS grows with the graph size and that, in the base program, it uses considerably more memory than BFS as the number of nodes increases. For 200 vertices and 50% fill, DFS reached 30.207 ms and 86.64 KB median memory. The node-scaling behavior is represented in Figure 2.1.

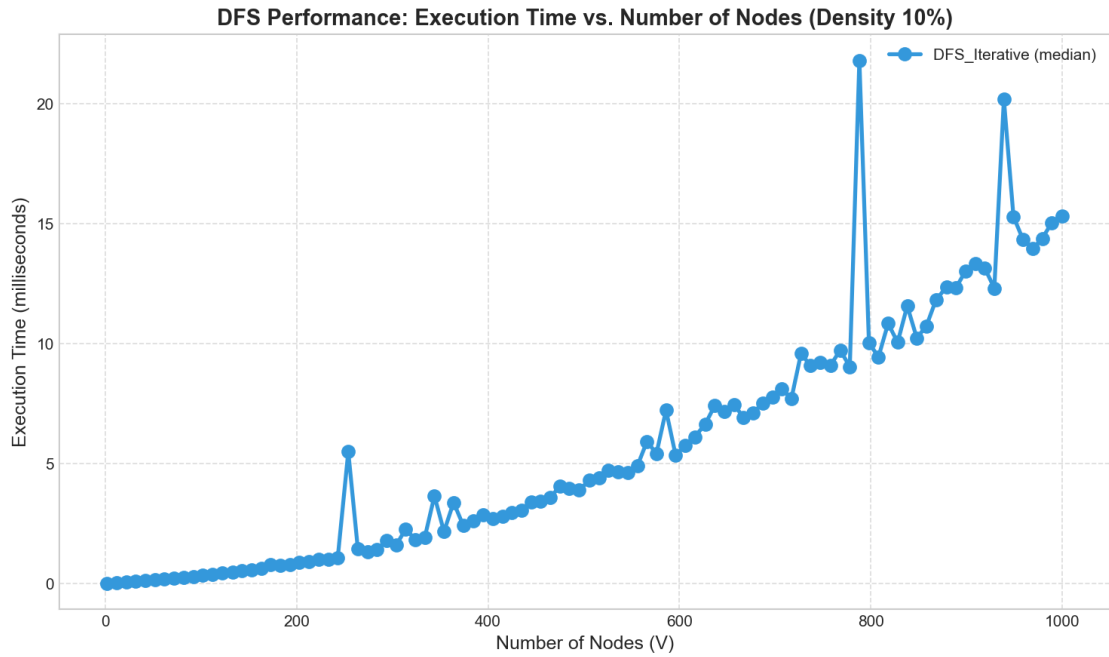


Figure 2.1 – DFS performance depending on the number of nodes

In Figure 2.1 the growth of execution time relative to the number of nodes can be observed directly. The increase is stronger on denser configurations because a larger number of edges must be processed.

Conclusion

DFS is effective for graph traversal and follows the expected $O(V + E)$ trend. However, the implementation details matter: the optimized version remains more stable on large and dense graphs, while the base version can consume significantly more memory.

2.2 Breadth First Search

The BFS implementation uses queue-based traversal. It visits all neighbors on the current level before moving to the next depth level, which makes it suitable for shortest paths in unweighted graphs.

Algorithm Description

The BFS method follows the idea shown in the next pseudocode.

```

BFS(graph, start):
    mark start as visited
    enqueue start
    while queue is not empty:
        node = dequeue queue
        for each neighbor of node:
            if neighbor is not visited:
                mark neighbor as visited
                enqueue neighbor

```


Implementation

The optimized BFS variant uses efficient queue operations and fast visited checks. The corresponding benchmark scripts compare it with DFS over both density-based and topology-based inputs.

Results

After running the BFS function for the generated graphs, the optimized topology benchmark produced the representative values shown in Table 2.2.

Table 2.2 – Optimized topology results at 1000 nodes

Graph type	Algorithm	Time (ms)	Memory (KB)	Edges
bipartite_graph	BFS_Optimized	13.490	48.57	75187.0
bipartite_graph	DFS_Optimized	41.326	659.13	75187.0
complete_graph	BFS_Optimized	96.790	49.60	499500.0
complete_graph	DFS_Optimized	224.583	4110.82	499500.0
grid_planar	BFS_Optimized	1.680	44.20	1936.0
grid_planar	DFS_Optimized	4.848	48.79	1936.0
tree	BFS_Optimized	2.200	44.16	999.0
tree	DFS_Optimized	4.122	43.05	999.0

In Table 2.2 it can be observed that BFS behaves very differently depending on the graph topology. On a tree of 1000 vertices, BFS needed only 2.200 ms, while on a complete graph with 499500 edges it needed 96.790 ms. This confirms that the edge count has a decisive effect on the execution time. The node-based BFS behavior is shown in Figure 2.2, and the combined analysis is shown in Figure 2.3.

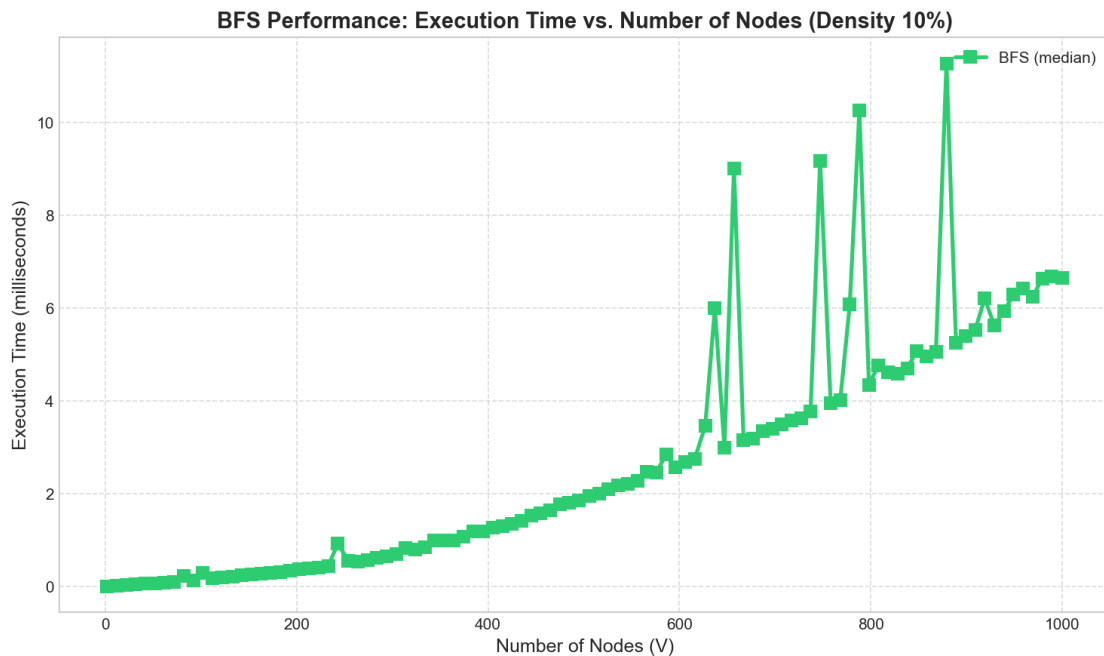


Figure 2.2 – BFS performance depending on the number of nodes

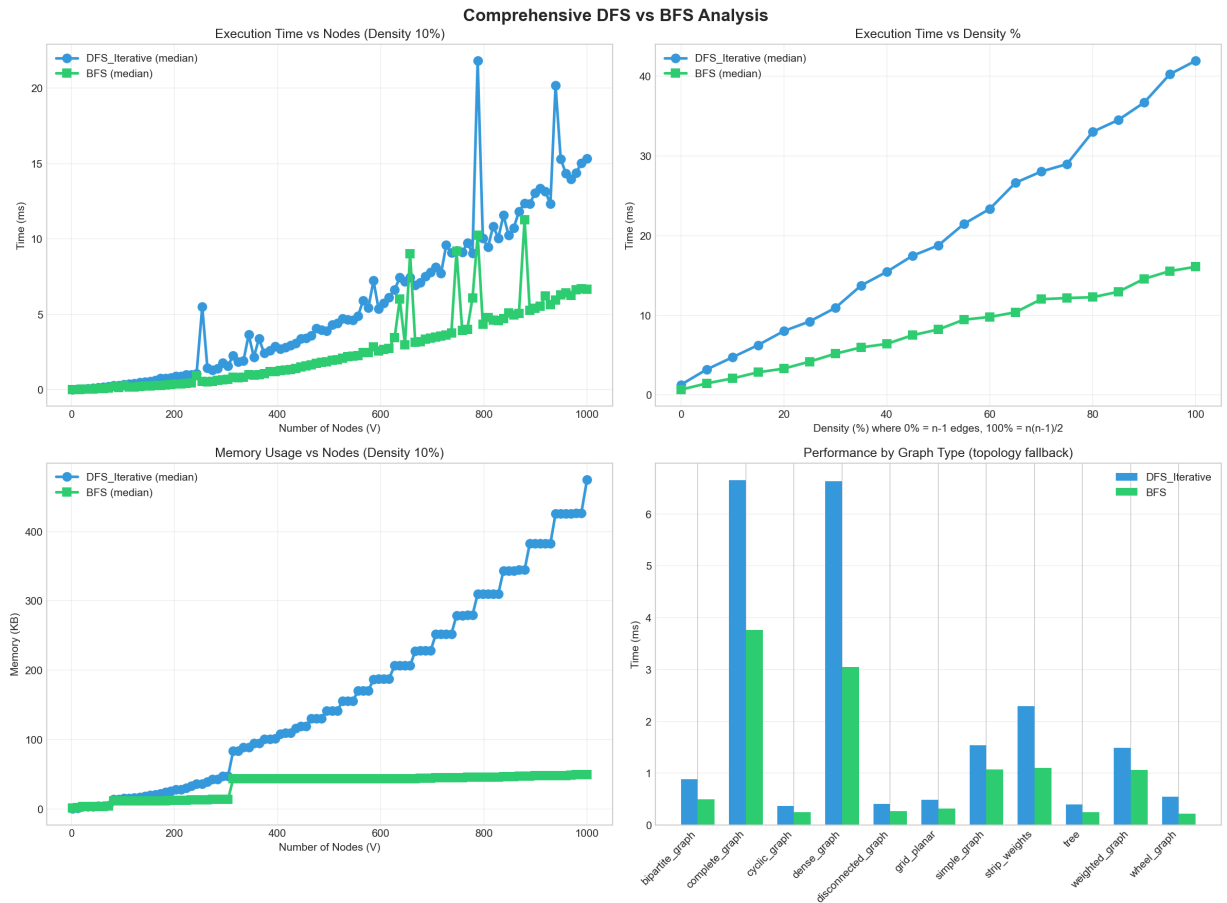


Figure 2.3 – Comprehensive DFS and BFS analysis

Figure 2.3 offers a combined perspective on the two algorithms. It shows execution time versus nodes, execution time versus density, memory usage versus nodes, and performance by graph type.

Conclusion

BFS proved to be the best average performer among the optimized variants in the generated Lab 3 summaries. It remained especially effective on sparse topologies such as trees and grid-planar graphs, while its running time increased substantially on complete graphs because of the much larger edge set.

CONCLUSION

The performed experiments confirm that both DFS and BFS follow the theoretical $O(V + E)$ tendency, but their practical behavior is strongly influenced by graph density, topology, and implementation details. Sparse structures such as trees and grid graphs remain inexpensive, while complete and near-complete graphs increase the cost considerably.

The four-version organization of the laboratory was useful because it separated two ideas of classification: percentage of fill and graph topology. This produced a clearer empirical analysis than a single benchmark setup would have provided. Overall, BFS was the fastest average optimized variant, while DFS was more sensitive to the structure of the graph and to the choice of data structures used internally.

REFERENCES

REFERENCES

- [1] GeeksforGeeks. *Breadth First Search or BFS for a Graph* [online]. [cited 07.05.2026]. Available: [geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph](https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph).
- [2] GeeksforGeeks. *Depth First Search or DFS for a Graph* [online]. [cited 07.05.2026]. Available: [geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph](https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph).
- [3] YouTube. *Graph Traversal Algorithms* [video online]. [cited 07.05.2026]. Available: [youtube.com/watch?v=zaBhtODEL0w](https://www.youtube.com/watch?v=zaBhtODEL0w).
- [4] GitHub Repository. *Lab 3 source folder* [online]. [cited 07.05.2026]. Available: github.com/Prince-of-Nothing/aa-course-repo/tree/main/Lab3.